

12

Interfacing with GraphQL on the Frontend Using Apollo Client

After successfully implementing a GraphQL backend using Apollo Server in the previous chapter, we are now going to interface with our new GraphQL API on the frontend using Apollo Client. Apollo Client is a library that makes it easier and more convenient to interact with GraphQL APIs. We'll start by replacing the fetching of the post list with GraphQL queries, then resolve the author usernames without needing extra queries, showing the power of GraphQL. Next, we'll add variables to the query to allow setting filters and sorting options. Finally, we'll learn how to use mutations on the frontend.

In this chapter, we are going to cover the following main topics:

- Setting up Apollo Client and making our first query
- Using variables in GraphQL queries
- Using mutations on the frontend

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing For Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If there is an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch12>.

The CiA video for this chapter can be found at:

https://youtu.be/GI_5i9DR_xA.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

Setting up Apollo Client and making our first query

Before we can get started making GraphQL queries on the frontend, we first need to set up Apollo Client. **Apollo Client** is the frontend counterpart to Apollo Server, which we have already been using on the backend. While it is not required to use Apollo Client (we could also simply make a `POST` request to the `/graphql` endpoint), Apollo Client makes interacting with GraphQL much easier and more convenient. It also includes additional features, such as caching, out of the box.

Follow these steps to set up Apollo Client:

1. Copy the existing `ch11` folder to a new `ch12` folder, as follows:

```
$ cp -R ch11 ch12
```

2. Open the `ch12` folder in VS Code.
3. Install the `@apollo/client` and `graphql` dependencies:

```
$ npm install @apollo/client@3.9.5 graphql@16.8.1
```

4. Edit `.env` and add a new environment variable, pointing to the endpoint for our GraphQL server:

```
VITE_GRAPHQL_URL="http://localhost:3001/graphql"
```

5. Edit **src/App.jsx** and import **ApolloClient**, **InMemoryCache**, and **ApolloProvider** from the **@apollo/client** package:

```
import { ApolloProvider } from '@apollo/client/react/index.js'
import { ApolloClient, InMemoryCache } from '@apollo/client/core/index.js'
```

At the time of writing, there is an issue with ESM imports in Apollo Client, so we need to import directly from the **index.js** files.

6. Create a new instance of Apollo Client, pointing to the GraphQL endpoint and using **InMemoryCache**:

```
const apolloClient = new ApolloClient({
  uri: import.meta.env.VITE_GRAPHQL_URL,
  cache: new InMemoryCache(),
})
```

7. Adjust the **App** component to add **ApolloProvider**, providing the Apollo Client context to our whole app:

```
export function App({ children }) {
  return (
    <HelmetProvider>
      <ApolloProvider client={apolloClient}>
        <QueryClientProvider client={queryClient}>
          <AuthContextProvider>{children}</AuthContextProvider>
        </QueryClientProvider>
      </ApolloProvider>
    </HelmetProvider>
  )
}
```

8. We are also going to create a GraphQL config file now so that the VS Code GraphQL extension can autocomplete and validate queries for us. Create a new **graphql.config.json** file in the root of the project, with the following contents:

```
{
  "schema": "http://localhost:3001/graphql",
  "documents": "src/api/graphql/**/*.{js,jsx}"
}
```

The **schema** defines the URL to the GraphQL endpoint, and **documents** defines where to find files that contain GraphQL queries. We are going

to put the GraphQL queries in the **src/api/graphql/** folder later.

9. Make sure Docker and the database container are running, then start the backend, as follows:

```
$ cd backend/  
$ npm run dev
```

Keep the backend running throughout this chapter, so that the GraphQL extension can access the GraphQL endpoint.

10. Restart the VS Code GraphQL extension. You can do so by accessing the VS Code command palette (*Ctrl + Shift + P* on Windows/Linux and *Cmd + Shift + P* on macOS) and typing in **GraphQL: Manual Restart**.

Querying posts from the frontend using GraphQL

Now that Apollo Client is set up and ready to be used, let's define our first GraphQL query: a simple query to fetch all posts.

Follow these steps to define the query and use it in our app:

1. Create a new **src/api/graphql/** folder, where we will put our GraphQL queries.
2. Inside this folder, create a new **src/api/graphql/posts.js** file.
3. In the **src/api/graphql/posts.js** file, import the **gql** function from **@apollo/client**:

```
import { gql } from '@apollo/client/core/index.js'
```

4. Define a new **GET_POSTS** query, which retrieves all the relevant properties for a post (except the author, which will come later):

```
export const GET_POSTS = gql`  
  query getPosts {  
    posts {  
      id  
      title  
      contents  
      tags  
      updatedAt  
      createdAt  
    }  
  }  
`
```

```
}  
}  
,
```

You should see that the GraphQL extension is offering us autocomplete options for the types we defined in our backend! If we enter the wrong field name, it will also warn us that this field does not exist on the type.

5. Edit `src/pages/Blog.jsx` and import the `useQuery` hook from `@apollo/client`:

```
import { useQuery as useGraphQLQuery } from '@apollo/client/react/index.js'
```

We renamed the `useQuery` hook from Apollo Client to `useGraphQLQuery` to avoid confusion with the `useQuery` hook from TanStack React Query.

6. Import the previously defined `GET_POSTS` query:

```
import { GET_POSTS } from '../api/graphql/posts.js'
```

7. Remove the imports to `useQuery` and `getPosts`:

```
import { useQuery } from '@tanstack/react-query'  
import { getPosts } from '../api/posts.js'
```

8. Remove the existing `useQuery` hook:

```
const postsQuery = useQuery({  
  queryKey: ['posts', { author, sortBy, sortOrder }],  
  queryFn: () => getPosts({ author, sortBy, sortOrder }),  
})  
const posts = postsQuery.data ?? []
```

9. Replace it with the following hook:

```
const postsQuery = useGraphQLQuery(GET_POSTS)  
const posts = postsQuery.data?.posts ?? []
```

10. Make sure you are in the root of the project, then run the frontend as follows:

```
$ npm run dev
```

Now, open the frontend on **http://localhost:5173/** and you will see that the post titles are properly shown. However, the links to posts do not work and there is an error in the console. There is a slight difference in the results from GraphQL and the REST API: the REST API returns the ID of posts as an **_id** property, while GraphQL returns them as an **id** property.

Let's adjust our code to accommodate this change now:

1. Edit **src/components/Post.jsx** and change the **_id** prop to **id**:

```
export function Post({
  title,
  contents,
  author,
  id,
```

2. Also, update the variable name where it is used:

```
<Link to={` /posts/${id}/${slug(title)}`}>
```

3. Make sure to update **propTypes** as well:

```
Post.propTypes = {
  title: PropTypes.string.isRequired,
  contents: PropTypes.string,
  author: PropTypes.string,
  id: PropTypes.string.isRequired,
```

4. Now that the prop is changed, edit **src/pages/ViewPost.jsx** and pass in the new prop, as follows:

```
{post ? (
  <Post {...post} id={postId} fullPost />
) : (
  `Post with id ${postId} not found.`
)}
```

After saving all files, the frontend should refresh and properly render the list of all posts with working links. Now all that's left to do to restore the original functionality is to show the author usernames.

Resolving author usernames in a single query

Instead of resolving each author username separately, we can now get them all at once in a single query, thanks to the power of GraphQL! Let's make use of this power to refactor our code a bit to make it simpler and improve the performance:

1. Start by editing the GraphQL query in **src/api/graphql/posts.js**, adding the **author.username** field, as follows:

```
export const GET_POSTS = gql`
  query getPosts {
    posts {
      author {
        username
      }
    }
  }
`
```

2. Then, edit the **src/components/User.jsx** component. *Replace* the whole component with the following, simpler component:

```
import PropTypes from 'prop-types'
export function User({ username }) {
  return <b>{username}</b>
}
User.propTypes = {
  username: PropTypes.string.isRequired,
}
```

It is not necessary to fetch the user info here anymore, as we can directly display the username from the GraphQL response.

3. Next, edit **src/components/Post.jsx** and pass the whole **author** object to the **User** component, as follows:

```
Written by <User {...author} />
```

4. We also need to adjust **propTypes** now to accept a full **author** object for the **Post** component, instead of a user ID:

```
author: PropTypes.shape(User.propTypes),
```

5. Edit **src/pages/ViewPost.jsx** and pass the whole **author** object to the **Post** component:

```
<Post {...post} id={postId} author={userInfo} fullPost />
```

Thankfully, we are already resolving the username for the meta tags on this page, so we do not need to make an additional query here either.

6. However, in **Header**, we do need to make an additional query to resolve the username when a user is logged in.
Edit **src/components/Header.jsx** and import the **useQuery** hook and the **getUserInfo** API function:

```
import { useQuery } from '@tanstack/react-query'  
import { getUserInfo } from '../api/users.js'
```

7. Then, adjust the component to get the user ID from the token (the **sub** field of the JWT) and make a query for the user info:

```
export function Header() {  
  const [token, setToken] = useAuth()  
  const { sub } = token ? jwtDecode(token) : {}  
  const userInfoQuery = useQuery({  
    queryKey: ['users', sub],  
    queryFn: () => getUserInfo(sub),  
    enabled: Boolean(sub),  
  })  
  const userInfo = userInfoQuery.data
```

8. Lastly, we check whether we were able to resolve the query for user info (instead of just checking for **token**). If so, we pass the user info to the **User** component:

```
if (token && userInfo) {  
  return (  
    <nav>  
      Logged in as <User {...userInfo} />  
    </nav>  
  )  
}
```

We also removed the token decoding here, like we already did earlier.

Now we are using GraphQL to fetch the list of posts and resolve the author usernames in one single request! However, the filters and sorting do

not work anymore, as we are not passing this information to the GraphQL query yet.

In the next section, we are going to introduce variables for filtering and sorting our GraphQL queries.

Using variables in GraphQL queries

To add support for filters and sorting, we need to add variables to our GraphQL query. We can then fill in these variables when executing the query.

Follow these steps to add variables to the query:

1. Edit `src/api/graphql/posts.js` and adjust the query to accept an `$options` variable:

```
export const GET_POSTS = gql`
  query getPosts($options: PostsOptions) {
```

2. Then, pass the `$options` variable to the `posts` resolver, for which we already implemented an `options` argument in the previous chapter:

```
posts(options: $options) {
```

3. Now, we just need to pass the options when executing the query. Edit `src/pages/Blog.jsx` and pass the variable, as follows:

```
const postsQuery = useGraphQLQuery(GET_POSTS, {
  variables: { options: { sortBy, sortOrder } },
})
```

4. Go to the blog frontend and change the sort order to ascending to see the variable in action!

Using fragments to reuse parts of queries

Now that sorting is working, we just need to add filtering by author. To do this, we need to add a second query for `postsByAuthor`. As you can imag-

ine, this query should return the same fields as the **posts** query. We can make use of a fragment to reuse the fields for both queries, as follows:

1. Edit **src/api/graphql/posts.js** and define a new fragment in GraphQL that contains all the fields that we need from a post:

```
export const POST_FIELDS = gql`
  fragment PostFields on Post {
    id
    title
    contents
    tags
    updatedAt
    createdAt
    author {
      username
    }
  }
`
```

The fragment is defined by giving it a name (**PostFields**) and specifying which type it can be used on (**on Post**). Then, all fields from the specified type can be queried in the fragment.

2. To use the fragment, we first have to include its definition in the **GET_POSTS** query:

```
export const GET_POSTS = gql`
  ${POST_FIELDS}
  query getPosts($options: PostsOptions) {
```

3. Now, instead of listing all fields manually, we can use the fragment:

```
    posts(options: $options) {
      ...PostFields
    }
  }
`
```

The syntax for using a fragment is like object destructuring in JavaScript, where all properties defined in an object are spread into another object.

NOTE

*Sometimes the VS Code GraphQL extension needs to be restarted to be able to detect fragments properly. You can do so by accessing the VS Code command palette (Ctrl + Shift + P on Windows/Linux and Cmd + Shift + P on macOS) and typing in **GraphQL: Manual Restart**.*

4. Next, we define a second query, where we query posts by author, and get all necessary fields with the fragment:

```
export const GET_POSTS_BY_AUTHOR = gql`
  ${POST_FIELDS}
  query getPostsByAuthor($author: String!, $options: PostsOptions) {
    postsByAuthor(username: $author, options: $options) {
      ...PostFields
    }
  }
`
```

We defined the **\$author** variable as required for this query (by using an exclamation mark after the type). We need to do this because the **postsByAuthor** field also requires the first argument (**username**) to be set.

5. Edit **src/pages/Blog.jsx** and import the newly defined query:

```
import { GET_POSTS, GET_POSTS_BY_AUTHOR } from '../api/graphql/posts.js'
```

6. Then, adjust the hook to use the **GET_POSTS_BY_AUTHOR** query if **author** is defined:

```
const postsQuery = useGraphQLQuery(author ? GET_POSTS_BY_AUTHOR : GET_POSTS,
```

7. Pass the **author** variable to the query:

```
  variables: { author, options: { sortBy, sortOrder } },
))
```

8. Lastly, we need to adjust how we select the results because the **postsByAuthor** field from the **GET_POSTS_BY_AUTHOR** query will return the results in **data.postsByAuthor**, while the **GET_POSTS** query uses the **posts** field, which returns results in **data.posts**. As there is no case where both fields are returned at once, we can simply do the following:

```
const posts = postsQuery.data?.postsByAuthor ?? postsQuery.data?.posts ?? []
```

9. Go to the frontend and try filtering by author. The filter works again now!

As we can see, fragments are very useful for reusing the same fields for multiple queries! Now that our post list is fully refactored to use GraphQL, let's move on to using mutations on the frontend, allowing us to migrate the signup, login, and create post functionalities to GraphQL.

Using mutations on the frontend

As we learned in the previous chapter, mutations in GraphQL are used to change the state of the backend (similar to **POST** requests in REST). We are now going to implement mutations for signing up and logging in to our app.

Follow these steps:

1. Create a new **src/api/graphql/users.js** file and import **gql**:

```
import { gql } from '@apollo/client/core/index.js'
```

2. Then, define a new **SIGNUP_USER** mutation, which takes a username and a password and calls the **signupUser** mutation field:

```
export const SIGNUP_USER = gql`
  mutation signupUser($username: String!, $password: String!) {
    signupUser(username: $username, password: $password) {
      username
    }
  }
`
```

3. Edit **src/pages/Signup.jsx** and *replace* the current **useMutation** hook from TanStack React Query with the one from Apollo Client. As we did before for **useQuery**, we are also going to rename this hook to **useGraphQLMutation** to avoid confusion:

```
import { useMutation as useGraphQLMutation } from '@apollo/client/react/index.
```

4. Additionally, *replace* the import of the **signup** function with an import of the **SIGNUP_USER** mutation:

```
import { SIGNUP_USER } from '../api/graphql/users.js'
```

5. *Replace* the existing mutation hook with the following:

```
const [signupUser, { loading }] = useGraphQLMutation(SIGNUP_USER, {
  variables: { username, password },
  onCompleted: () => navigate('/login'),
  onError: () => alert('failed to sign up!'),
})
```

As can be seen, the Apollo Client mutation hook has a slightly different API than the TanStack React Query mutation hook. It returns an array with a function to call the mutation, and an object with the loading state, error state, and data. Similar to the **useGraphQLQuery** hook, it also accepts the mutation as the first argument and an object with variables as the second argument. Moreover, the **onSuccess** function is called **onCompleted** in Apollo Client.

6. Change the **handleSubmit** function as follows:

```
const handleSubmit = (e) => {
  e.preventDefault()
  signupUser()
}
```

7. Lastly, change the submit button as follows:

```
<input
  type='submit'
  value={loading ? 'Signing up...' : 'Sign Up'}
  disabled={!username || !password || loading}
/>
```

Now the signup functionality is successfully migrated to GraphQL. Next, let's migrate the login functionality.

Migrating login to GraphQL

Refactoring the login functionality to GraphQL is very similar to the signup functionality, so let's quickly go through the steps:

1. Edit **src/api/graphql/users.js** and define a mutation for logging in:

```
export const LOGIN_USER = gql`
  mutation loginUser($username: String!, $password: String!) {
    loginUser(username: $username, password: $password)
  }
`
```

2. Edit **src/pages/Login.jsx** and *replace* the imports to TanStack React Query and the **login** function with the following:

```
import { useMutation as useGraphQLMutation } from '@apollo/client/react/index.'
import { LOGIN_USER } from '../api/graphql/users.js'
```

3. Update the hook as well:

```
const [loginUser, { loading }] = useGraphQLMutation(LOGIN_USER, {
  variables: { username, password },
  onCompleted: (data) => {
    setToken(data.loginUser)
    navigate('/')
  },
  onError: () => alert('failed to login!'),
})
```

4. Update the **handleSubmit** function:

```
const handleSubmit = (e) => {
  e.preventDefault()
  loginUser()
}
```

5. Finally, update the submit button:

```
<input
  type='submit'
  value={loading ? 'Logging in...' : 'Log In'}
```

```
disabled={!username || !password || loading}  
/>
```

Now that signup and login are using GraphQL mutations, let's move on to migrating the create post functionality to GraphQL.

Migrating create post to GraphQL

The create post functionality is a bit trickier to implement, as it requires us to be logged in (which means that we need to send the JWT header), and invalidate the post list queries, so that the list gets updated after creating a new post.

Now let's get started on implementing this with Apollo Client:

1. First, let's define the mutation. Edit **src/api/graphql/posts.js** and add the following code:

```
export const CREATE_POST = gql`  
  mutation createPost($title: String!, $contents: String, $tags: [String!]) {  
    createPost(title: $title, contents: $contents, tags: $tags) {  
      id  
      title  
    }  
  }  
`
```

For this mutation, we are going to use the response to get the **id** and **title** of the created post. We are going to make use of this data to show a link to the post upon successful creation.

2. Then, edit **src/components/CreatePost.jsx** and *replace* the TanStack React Query import with an import of the mutation hook:

```
import { useMutation as useGraphQLMutation } from '@apollo/client/react/index.
```

3. Also, import the **Link** component and **slug** function to show a link to the created post:

```
import { Link } from 'react-router-dom'  
import slug from 'slug'
```

4. Replace the import of the **createPost** function with imports of the **CREATE_POST** mutation and the **GET_POSTS** and **GET_POSTS_BY_AUTHOR** queries. We are going to use these query definitions to make Apollo Client re-fetch them for us later:

```
import {
  CREATE_POST,
  GET_POSTS,
  GET_POSTS_BY_AUTHOR,
} from '../api/graphql/posts.js'
```

5. Replace the existing query client and mutation hooks with the following GraphQL mutation, where we pass the **title** and **contents** variables:

```
const [createPost, { loading, data }] = useGraphQLMutation(CREATE_POST, {
  variables: { title, contents },
```

6. Next, we provide the JWT header as **context** to the mutation:

```
context: { headers: { Authorization: `Bearer ${token}` } } },
```

7. Then, we provide a **refetchQueries** option to the mutation, telling Apollo Client to re-fetch certain queries after the mutation was called:

```
refetchQueries: [GET_POSTS, GET_POSTS_BY_AUTHOR],
})
```

NOTE

As re-fetching after a mutation is a common operation, Apollo Client provides a simple way to do this in the mutation hook. Simply pass all queries that should be re-fetched there, and Apollo Client will take care of it.

8. Adjust the **handleSubmit** function:

```
const handleSubmit = (e) => {
  e.preventDefault()
  createPost()
}
```

9. Adjust the submit button:

```
<input
  type='submit'
  value={loading ? 'Creating...' : 'Create'}
  disabled={!title || loading}
/>
```

10. Lastly, we are going to change the success message, showing a link to the created post:

```
{data?.createPost ? (
  <>
    <br />
    Post{' '}
    <Link
      to={` /posts/${data.createPost.id}/${slug(data.createPost.title)}`}
    >
      {data.createPost.title}
    </Link>{' '}
    created successfully!
  </>
) : null}
```

Because of the way types and resolvers work in GraphQL, it easily allows us to access fields from the result of a mutation, the same way as if we were fetching a single post. For example, we could even tell GraphQL to fetch the username of the author of the created post here!

11. Try creating a new post, and you will see that the success message now shows a link to the created post, and the post list automatically re-fetches for us!

The following screenshot shows a new post being successfully created, showing the link to the new post in the success message, and the new post in the post list (automatically re-fetched by Apollo Client):

Title:

This post was created using GraphQL mutations!

Create

Post [New Post](#) created successfully!

Filter by:
author:

Sort By: / Sort Order:

[New Post](#)

Written by testuser

Figure 12.1: Creating a post using GraphQL mutations, with a re-fetching post list

Now that we have successfully implemented creating posts with GraphQL, our blog app is fully connected to our GraphQL server.

There are many more advanced concepts in GraphQL that we have not covered yet in this book, such as advanced re-fetching, subscriptions (getting real-time updates from the GraphQL server), error handling, suspense, pagination, and caching. The GraphQL chapters in this book only serve as an introduction to GraphQL.

If you wish to learn more about GraphQL and Apollo, I recommend checking out the extensive Apollo docs (<https://www.apollographql.com/docs/>), which contain detailed information and hands-on examples about using Apollo Server and Apollo Client.

Summary

In this chapter, we connected our previously created GraphQL backend to the frontend using Apollo Client. We started by setting up Apollo Client and making a GraphQL query to fetch all posts. Then, we improved the performance of the post list by fetching author usernames in a single request, leveraging the power of GraphQL.

Next, we introduced variables in our query and re-implemented sorting and filtering by author. We also introduced fragments in our queries to reuse the same fields. Lastly, we implemented GraphQL mutations in the frontend to sign up, log in, and create posts. We also learned about re-fetching queries in Apollo Client along the way and briefly mentioned advanced concepts of GraphQL and Apollo.

In the next chapter, [***Chapter 13***](#), *Building a Backend Based on Event-Driven Architecture Using Express and Socket.IO*, we are going to depart from traditional full-stack architectures and build a new app using a special kind of full-stack architecture: an event-based application.